

IEEE 754 Floating-Point Representation

Single Precision (32-bit)

Why Floating Point?

- Integers cannot represent fractions or very large/small numbers
- Floating point allows approximate representation of real numbers
- IEEE 754 is a universal standard used in CPUs and FPUs

32-bit Layout

Sign (1 bit)	Exponent (8 bits)	Mantissa (23 bits)
--------------	-------------------	--------------------

- Bias = 127
- Leading 1 is implicit for normalized numbers

Value Representation

IEEE 754 single precision represents numbers as:

$$(-1)^S \times (1 + \text{fraction}) \times 2^{(E-127)}$$

- S = Sign bit
- E = Stored exponent
- Fraction = Mantissa bits

Normalization

- Normalization forces the binary number into the form:

$$1.xxxxx \times 2^E$$

- Exactly one non-zero bit before the binary point
- Ensures unique representation and better precision

Why Bias is Used

- Exponents can be positive or negative
- Hardware prefers unsigned values
- Bias allows storing exponent as unsigned

$$\text{Stored exponent} = \text{Actual exponent} + 127$$

Example 1: Positive Number (85.125)

Convert decimal number 85.125 to IEEE 754 single precision.

Binary Conversion of 85.125

Integer part:

$$85_{10} = 1010101_2$$

Fractional part:

$$0.125_{10} = 0.001_2$$

Combined:

$$1010101.001_2$$

Normalization of 85.125

$$1010101.001_2 = 1.010101001_2 \times 2^6$$

- Actual exponent = +6
- Mantissa = 010101001

IEEE 754 Fields for 85.125

- Sign bit = 0 (positive)
- Exponent = $127 + 6 = 133 = 10000101_2$
- Mantissa (23 bits):

01010100100000000000000

Final Representation of 85.125

0 10000101 010101001000000000000000

Hexadecimal:

42AA4000

Example 2: Negative Number with Negative Exponent

Convert decimal number -0.15625 to IEEE 754 single precision.

Binary Conversion of 0.15625

$$0.15625_{10} = 0.00101_2$$

Normalization of 0.15625

$$0.00101_2 = 1.01_2 \times 2^{-3}$$

- Actual exponent = -3
- Mantissa = 01

IEEE 754 Fields for -0.15625

- Sign bit = 1 (negative)
- Exponent = $127 - 3 = 124 = 01111100_2$
- Mantissa (23 bits):

01000000000000000000000

Final Representation of -0.15625

1 01111100 010000000000000000000000

Hexadecimal:

BE200000

Does the CPU Convert Decimal to Binary?

- **No — CPUs do not understand decimal numbers**
- RISC-V and all CPUs operate only on binary bit patterns
- Decimal-to-binary conversion is done outside the CPU

Decimal to Binary Conversion Happens at Compile Time

Example C code:

```
float x = 3.785;
```

What happens:

- 1 Compiler converts 3.785 into IEEE 754 binary format
- 2 Encodes the value as 0x40723D71
- 3 CPU simply loads and operates on the binary bits

Key Insight

- The multiply-by-2 method is for humans
- Hardware never performs decimal conversion
- Floating-point units manipulate sign, exponent, and mantissa directly

Special Cases

- Exponent all 0s $\rightarrow$ zero or subnormal numbers
- Exponent all 1s $\rightarrow$ infinity or NaN
- Subnormal numbers do not use implicit leading 1

Summary

- IEEE 754 defines floating-point representation
- Uses sign, biased exponent, and mantissa
- Normalization ensures unique representation
- Decimal-to-binary conversion occurs at compile time